DuocUC | ONLINE



Desarrollo Backend III

Informe técnico Banco XYZ

Lucas Avilés

Docente Alonso Esteban Castillo Pizarro

Evaluación Final Transversal PBY2203

11 de octubre de 2026

Índice

1 Proyecto y necesidades....3

2 Arquitectura e integración....4

3 Migración con Spring Batch....5

4 Reglas financieras y calidad de datos....6

5 Datos y resultados batch....7

6 BFF y contratos por canal....8

7 Microservicios y consistencia de pagos....9

8 Seguridad distribuida....10

9 Resiliencia y mensajería....11

10 Contenedores y escalado....12

11 Resultados y próximos pasos....13

12 Referencias técnicas....14

1 Proyecto y necesidades

Banco XYZ necesita modernizar procesos COBOL y Shell que concentran transacciones, intereses, estados de cuenta y perfiles en un monolito. La solución implementada separa responsabilidades y conserva la trazabilidad de los datos de entrada. El resultado es un proyecto ejecutable con Spring Batch, tres BFF y servicios de Cuentas, Pagos y Clientes, integrados mediante Spring Cloud, OAuth2, Kafka y JMS.

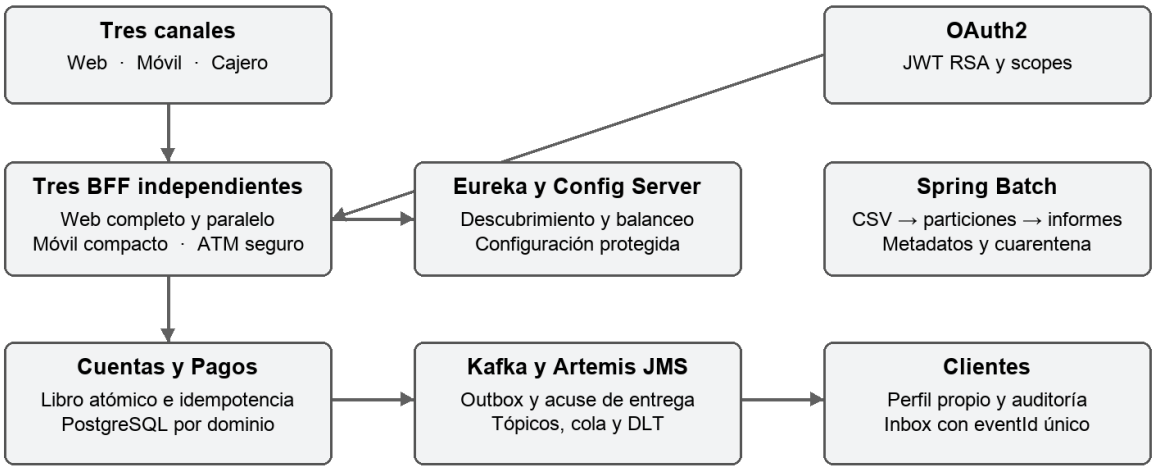
Requerimientos del negocio

Continuidad e integridad monetaria. Un pago debe producir un movimiento único, incluso si falla una llamada después del commit. Por eso Cuentas mantiene el libro transaccional y Pagos registra una intención durable antes de reconciliarla. Se evita informar como realizado un movimiento cuyo resultado aún no se conoce.
Atención eficiente por canal. Web necesita una vista agregada; móvil debe reducir datos transmitidos; el cajero requiere autorización, PIN y confirmación de saldo. Cada BFF es un proceso independiente, con contrato y permisos específicos.
Auditoría y capacidad de crecimiento. El banco debe explicar cada transformación y cada evento. Se conservan CSV originales, metadatos batch, filas rechazadas e identificadores de pagos y eventos. Las APIs sin estado local pueden replicarse sobre persistencia compartida.

Procesos de modernización

La migración comprende cinco frentes: reescritura de tres procesos legacy con Spring Batch; separación del monolito en dominios de negocio; BFF para los tres canales; seguridad distribuida mediante Spring Security OAuth2 y configuración Cloud; integración asíncrona con Kafka, complementada por JMS. La implantación puede avanzar por proceso y canal, usando reconciliación de resultados antes de retirar cada flujo legacy.

2 Arquitectura e integración



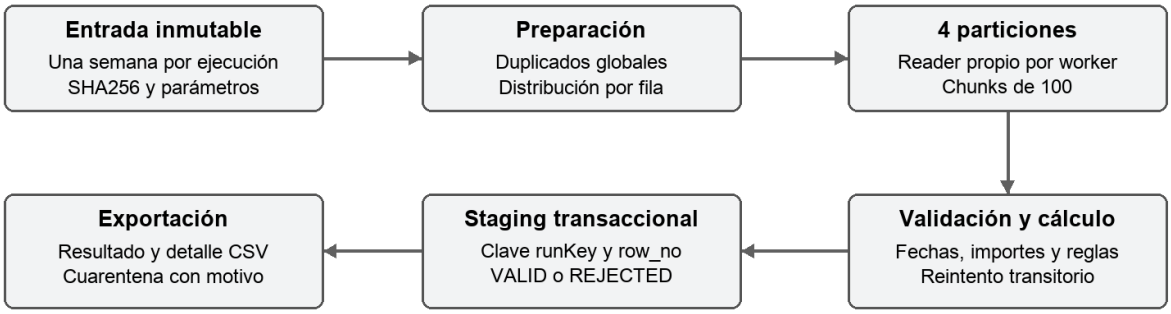
Los BFF consultan servicios por nombre lógico. Eureka mantiene las instancias y Spring Cloud LoadBalancer resuelve cada llamada. Config Server entrega configuración común y por módulo, protegido con credenciales de infraestructura. OAuth2 firma JWT RSA; cada API valida emisor, audiencia, expiración y scopes.

Cuentas, Pagos, Clientes y Batch usan bases PostgreSQL distintas con usuarios independientes. Ningún servicio consulta tablas de otro dominio. El único agregado que modifica saldos es Cuentas. Pagos lo invoca con una credencial técnica que sólo permite postings.

El broker no sustituye la transacción financiera. La confirmación del pago y su outbox se guardan juntas; la publicación ocurre después. Clientes consume eventos y los registra por eventId, por lo que una redelivery no duplica la auditoría.

El batch opera sobre los archivos oficiales y genera informes y cuarentena. Sus resultados no se cargan arbitrariamente como saldos online, porque el dataset carece de identidad OAuth2 y de un saldo inicial anual verificable.

3 Migración con Spring Batch



Los jobs dailyTransactions, monthlyInterest y annualStatements encadenan preparación, un step particionado de lectura transformación escritura y exportación. La preparación distribuye filas en cuatro archivos y detecta duplicados globales. Cada partición tiene un FlatFileItemReader con alcance de step y checkpoint propio.

Se procesan chunks de 100 dentro de transacciones PostgreSQL. El procesador produce VALID o REJECTED y registra el motivo. Los errores temporales de acceso a datos se reintentan tres veces con espera exponencial de 200 a 2.000 ms. Un fallo estructural detiene el job y conserva su estado para reejecución.

JobRepository persiste instancias, ejecuciones y checkpoints. La identidad contiene tipo, semana, período, tasas, workers y hash del archivo. Repetir una instancia completada devuelve 409. El relanzamiento automático mantiene parámetros y permite hasta tres reinicios; una modificación del archivo impide continuar una instancia anterior.

El staging tiene clave única runKey y row_no. Al repetir un chunk, su escritura reemplaza la misma fila. La salida pública se habilita sólo al completar el job. El número de workers es configurable entre 1 y 16; los readers no se comparten entre hilos.

4 Reglas financieras y calidad de datos

Transacciones diarias

Los importes deben ser positivos y tener como máximo dos decimales. Sólo se admiten crédito y débito. Se validan fechas estrictamente y se normalizan cuatro formatos conocidos. Los registros inválidos van a cuarentena; un importe de 3.000 o más se marca HIGH_VALUE y se cuenta como alerta en el resumen por fecha y tipo. El CSV contiene varios días, por lo que un job produce el reporte de cada día del año seleccionado.

Intereses mensuales

El archivo se denomina intereses_trimestrales.csv, pero la actividad exige intereses mensuales. Se toma su saldo como base del período y se aplica saldo × tasa anual / 12 con BigDecimal, HALF_UP y dos decimales. Las tasas académicas se reciben como parámetros: 0,03 para ahorro, 0,12 para préstamo y 0,08 para hipoteca. Se valida edad entre 18 y 100 y no se selecciona silenciosamente un saldo entre identificadores duplicados.

Cuenta	Saldo	Tipo	Interés mensual
101	5.000	Ahorro	12,50
102	8.000	Préstamo	80,00
105	7.000	Hipoteca	46,67

Estados anuales

Se agrupa por cuenta y se calculan depósitos, egresos y movimiento neto. Retiro, compra y pago se transforman a débito por su valor absoluto; un depósito negativo es inválido. Se exige descripción. La cuenta 101 del escenario base registra 1.000 de depósito y 500 de retiro: movimiento neto 500. No se presenta este valor como saldo final, porque no hay saldo inicial en el CSV.

5 Datos y resultados batch

Se utilizaron las tres carpetas del repositorio fin_legacy_data, commit 1ca7c0432319a1fc30f9469078f0cabf7081579f. Los archivos originales permanecen intactos. Cada semana se considera un escenario independiente para no mezclar snapshots ni duplicar movimientos al concatenar ejercicios. Las nueve ejecuciones finalizaron COMPLETED.

Semana	Proceso	Leídas	Válidas	Rechazadas
1	Diario	10	8	2
1	Intereses	8	8	0
1	Anual	9	8	1
2	Diario	10	7	3
2	Intereses	8	6	2
2	Anual	9	7	2
3	Diario	1000	392	608
3	Intereses	1000	0	1000
3	Anual	1000	626	374

La semana 3 de intereses contiene reiteraciones de los mismos identificadores. Ante balances incompatibles y sin una fecha de snapshot por registro, el sistema rechaza todas las ocurrencias de la cuenta. Los 1.000 rechazos expresan una decisión conservadora de integridad; no un error técnico del job. La corrección requiere resolver el dato de origen.

La evidencia incluye resultado.csv, detalle.csv y rechazados.csv por escenario. El registro original y su número de fila permiten reproducir el motivo de rechazo. Los formatos alternativos de fecha se normalizan sólo si representan un día válido; 2024-13-01 y 31/02/2024 no se corrigen automáticamente.

No se dispone del programa COBOL, tasas contractuales ni salida certificada del mainframe. La equivalencia comprobada cubre reglas explícitas y casos numéricos de referencia del dataset. Una migración real necesita ejecución paralela contra el legacy y aprobación de diferencias por el negocio.

6 BFF y contratos por canal

Web y móvil

BFF Web consulta perfil, cuentas y pagos mediante tareas concurrentes en un pool limitado. El contrato incluye los datos completos y fecha de generación. Si una dependencia temporalmente falla, esa sección indica indisponibilidad; las demás pueden responder. Los errores de autorización se propagan, en lugar de convertirlos en datos vacíos.
BFF Móvil devuelve identificador y saldo. Evita enviar datos personales, tipos de cuenta y el historial completo. En la demostración se comparó el tamaño de las respuestas JSON; el contrato móvil fue menor que el dashboard web. Esto reduce ancho de banda sin alterar el saldo consultado.

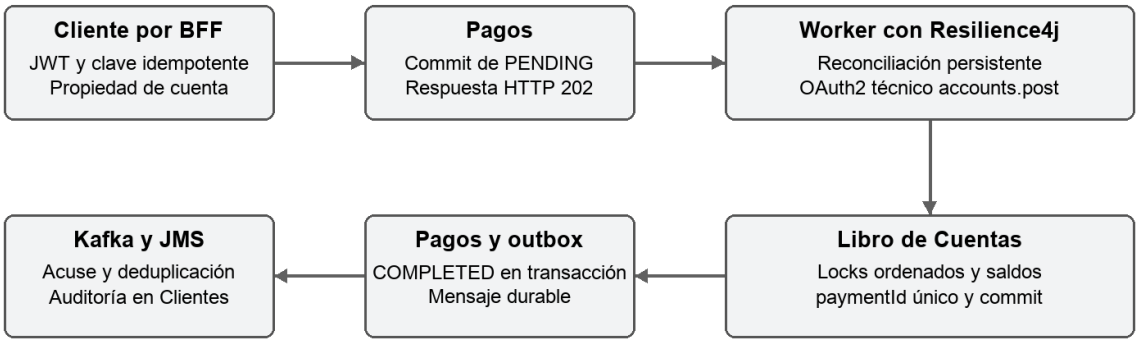
Cajero y operaciones críticas

BFF ATM exige un JWT con channel=atm, scope de lectura o escritura según la operación y PIN local. El saldo se confirma en Cuentas. El retiro tiene límite de 500 por solicitud, exige Idempotency-Key y sólo admite WITHDRAWAL. Ante caída de Cuentas responde indisponibilidad, sin inventar un saldo o confirmar dinero entregado.

Actor	Caso de uso	Resultado
Cliente web	Consultar dashboard y transferir	Vista agregada y pago pendiente
Cliente móvil	Consultar y ordenar un pago	Datos compactos y seguimiento
Cliente cajero	Consultar saldo y retirar	PIN, límite y confirmación
Operador	Ejecutar job y revisar eventos	Metadatos, CSV y auditoría

Cada BFF se construye y ejecuta como contenedor independiente. El JWT se propaga al dominio para conservar identidad y scopes. El claim de canal debe coincidir; un token web presentado al BFF móvil fue rechazado con HTTP 401.

7 Microservicios y consistencia de pagos



Cuentas ofrece apertura, mantenimiento y cierre con saldo cero. La aplicación de postings bloquea las cuentas en orden por identificador y ejecuta el débito y crédito en una sola transacción. Se valida propiedad, estado abierto, destino y fondos. El libro tiene paymentId único y huella de contenido, evitando reutilización inconsistente.

Pagos acepta la intención y responde 202 con PENDING. Un worker reclama registros mediante FOR UPDATE SKIP LOCKED y solicita el posting con credenciales accounts.post. Si Cuentas confirma, el pago pasa a COMPLETED y se escribe su evento en la misma transacción de Pagos. Un rechazo de negocio definitivo produce REJECTED. Un fallo de transporte conserva PENDING y se reconcilia.

Una caída entre el commit de Cuentas y el de Pagos no obliga a compensar un saldo desconocido: el worker repite el posting con el mismo paymentId y Cuentas devuelve la confirmación previa. Clientes administra el perfil propio y conserva la auditoría de eventos. La arquitectura evita transacciones distribuidas entre bases y hace explícita la consistencia eventual.

8 Seguridad distribuida

Spring Security protege cada endpoint; Spring Cloud distribuye los parámetros de seguridad y el descubrimiento. JWT usa firma RSA persistente, vida de cinco minutos, validación de issuer y audiencia bank-api. Se comprueban scopes y propiedad del recurso en el dominio, además de los permisos del BFF.

Identidad	Permisos principales
Web C001	Cuentas, perfil y pagos propios
Móvil C001	Lectura de cuentas y pagos propios
Cajero C001	Saldo y retiro con PIN
Caja C001	Depósito con deposits.write
Operador	batch.run, batch.read, ops.read
Worker Pagos	Sólo accounts.post
Cliente C002	Recursos propios; sin acceso a C001

La prueba sin token produjo 401; el acceso a una cuenta de otro cliente y la escritura con scope sólo de lectura produjeron 403. Una clave idempotente con contenido diferente produjo 409. Los depósitos exigen rol de caja: un cliente web no puede aumentar libremente su saldo. Config Server y Eureka exigen autenticación de infraestructura. Las credenciales se generan al preparar el entorno y quedan fuera de Git. Los contenedores de aplicación usan UID 10001. Caddy incorpora TLS para los BFF; la prueba local verificó su CA con --cacert y obtuvo respuesta UP.

Los clientes técnicos de la demostración tienen identidad fija. Para producción se requiere OIDC/PKCE por usuario, enrolamiento de tarjeta, gestión de secretos, rotación de claves y controles antifraude. Las llamadas internas y los brokers del perfil local usan una red Docker privada; varios hosts requieren TLS/mTLS y SASL/TLS.

9 Resiliencia y mensajería

Llamadas síncronas

Remote aplica Resilience4j con timeout de conexión de un segundo, lectura de dos segundos, hasta tres intentos ante transporte o HTTP 5xx, circuit breaker con mínimo de tres llamadas y espera de ocho segundos en estado abierto. El bulkhead limita doce llamadas concurrentes. Un error de negocio HTTP 4xx no se trata como fallo de disponibilidad.
Al pausar Cuentas, se observó un pago PENDING y el dashboard mostró indisponibilidad de la sección de cuentas. El circuito llegó a OPEN con tres llamadas fallidas. Tras restaurar el servicio, el pago quedó COMPLETED y el saldo disminuyó exactamente una unidad. La degradación permitió consultar el resto del dashboard durante la caída.

Eventos y entrega durable

Los pagos completados generan transactions.completed.v1. Los importes de 10.000 o más producen security.alerts.v1 y una alerta JMS bank.security.alerts. Cada mensaje contiene eventId, versión, fecha y referencia de pago. No contiene JWT, PIN ni secretos.
Outbox conserva mensajes dentro de la base del productor. La publicación espera el acuse del broker antes de marcar sent. Una caída posterior al acuse puede repetir un mensaje: Inbox usa eventId único para absorber esa duplicación. Esta estrategia garantiza entrega al menos una vez y efectos idempotentes, sin afirmar exactly once entre tecnologías distintas. Kafka tiene tres particiones, productores idempotentes, consumer groups y DLT tras reintentos del consumidor. JMS usa sesión transaccional y redelivery del broker. Los dos canales se comprobaron por entradas persistidas en Clientes. La outbox y los pagos pendientes pueden monitorearse mediante endpoints protegidos por ops.read.

10 Contenedores y escalado

Docker Compose coordina la infraestructura y las aplicaciones mediante dependencias de salud. El Dockerfile multietapa comparte la compilación Maven y produce una imagen final por módulo. Las APIs base sólo publican puertos en localhost; PostgreSQL, Kafka y Artemis no exponen puertos del host.

Se levantaron dos instancias de Cuentas, dos de Pagos y dos de Clientes. Eureka registró dos instancias UP por servicio. La configuración de escalado elimina puertos fijos de las APIs de negocio, conserva los BFF y permite que LoadBalancer descubra las réplicas.

Servicio	Instancias UP	Persistencia y reparto
Cuentas	2	Libro y saldos en PostgreSQL
Pagos	2	Trabajo reclamado con SKIP LOCKED
Clientes	2	Grupo Kafka y consumidores JMS

Con esas réplicas se ordenaron veinte transferencias concurrentes de una unidad. Las veinte finalizaron COMPLETED, el origen perdió veinte y el destino recibió veinte. Los bloqueos del libro impidieron pérdida de actualizaciones. El control idempotente se conserva al cambiar de instancia.

La escala batch se configura con workers y particiones locales. El perfil Compose replica aplicaciones en un host; una caída completa de esa máquina sigue siendo un punto de fallo. El procedimiento portable de despliegue se documenta para un servidor Linux con Caddy, DNS y TLS, sin presentar un despliegue remoto como realizado.

11 Resultados y próximos pasos

La solución completó los tres procesos batch sobre los nueve escenarios, distinguió datos válidos y rechazados y produjo salidas auditables. En el caso base, calculó 12,50 para ahorro de 5.000 al 3 % anual y movimiento neto anual de 500 en la cuenta 101. Los casos de fechas imposibles, signo, precisión y propiedad de recursos se comprobaron con pruebas automatizadas.

Las APIs ejecutaron apertura, mantenimiento, cierre con saldo cero y edición de perfil. La demostración produjo transferencias, depósito autorizado de 10.000 y retiro de 100; repetir una clave no repitió el débito. Kafka y JMS entregaron eventos persistidos y la publicación dejó su estado observable.

La prueba de fallo batch retiró permiso de escritura a una salida. La ejecución inicial falló, el planificador creó una reejecución y, al restaurar el permiso, una segunda reejecución completó el mismo job. Las ejecuciones 19, 20 y 21 mostraron retries 0, 1 y 2; el resultado final tuvo ocho filas válidas y una rechazada.

Respecto del legacy descrito, las mejoras implementadas son aislamiento de dominios, contratos por canal, permisos distribuidos, reejecución persistente y entrega asíncrona durable. No se dispone de una medición del mainframe para afirmar una reducción porcentual de latencia o costo.

Evolución

Los siguientes pasos son resolver la ambigüedad del origen de intereses; reconciliar resultados con salidas reales del COBOL; incorporar OIDC/PKCE y autenticación de tarjeta; versionar cambios de esquema; replicar bases y brokers; añadir TLS interno y trazas distribuidas; y medir throughput con volúmenes mayores. Antes de producción deben definirse políticas de retención, antifraude y recuperación del sitio completo.

12 Referencias técnicas

KariVillagran. fin_legacy_data. Repositorio de entrada de la actividad. Commit 1ca7c0432319a1fc30f9469078f0cabf7081579f. https://github.com/KariVillagran/fin_legacy_data

Spring. Spring Cloud. Compatibilidad de versiones, configuración, descubrimiento y balanceo. https://spring.io/projects/spring-cloud

Spring. Spring Batch Reference. Scaling and Parallel Processing. https://docs.spring.io/spring-batch/reference/scalability.html

Spring. Spring Security Reference. OAuth 2.0 Resource Server JWT. https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html

Spring. Spring Kafka Reference. Transactions. https://docs.spring.io/spring-kafka/reference/kafka/transactions.html

Docker. Use Compose in production. https://docs.docker.com/compose/how-tos/production/

Caddy. Automatic HTTPS. https://caddyserver.com/docs/automatic-https

Fuentes consultadas el 11 de octubre de 2026. El código fuente y las instrucciones de ejecución de esta entrega están disponibles en https://github.com/lcaviles1/banco-cloud-s8.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.